

**CRT**  
**ASSIGNMENT-1**

**Name: Subhapreet Patro**

**Roll No.: 2211CS010547**

**Section: I**

**1. Write a Java program to print the Climatic Season of your place.**

| <b>Months</b>                                  | <b>Season</b>  |
|------------------------------------------------|----------------|
| <b>12,1,2</b>                                  | <b>Winter</b>  |
| <b>3,4,5</b>                                   | <b>Spring</b>  |
| <b>6,7,8</b>                                   | <b>Summer</b>  |
| <b>9,10,11</b>                                 | <b>Monsoon</b> |
| <b>Anything else entered - "Invalid Month"</b> |                |

**Solution:**

```
import java.util.Scanner;

public class Q1 {
    public static void main(String[] args) {
        System.out.println("Subhapreet Patro:2211CS010547");
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter month number (1-12): ");
        int month = sc.nextInt();
        String season;
        switch (month) {
            case 12: case 1: case 2:
                season = "Winter";
                break;
            case 3: case 4: case 5:
                season = "Spring";
                break;
            case 6: case 7: case 8:
                season = "Summer";
                break;
            case 9: case 10: case 11:
                season = "Monsoon";
                break;
            default:
                season = "Invalid Month";
        }
        System.out.println("Season: " + season);
    }
}
```

```
}  
}
```

**Output:**

```
Subhapreet Patro:2211CS010547  
Enter month number (1-12): 10  
Season: Monsoon|
```

**2. Create a Java program that uses an enum to represent pizza sizes, defines a Pizza class with a constructor accepting a size, and implements a switch inside a method to print a customized message based on the selected size.**

**Solution:**

```
public class Q2 {  
    enum PizzaSize {  
        SMALL, MEDIUM, LARGE, EXTRA_LARGE  
    }  
  
    PizzaSize size;  
  
    public Q2(PizzaSize size) {  
        this.size = size;  
    }  
  
    public void printPizzaMessage() {  
        switch (size) {  
            case SMALL:  
                System.out.println("You ordered a Small pizza. Perfect for one!");  
                break;  
            case MEDIUM:  
                System.out.println("You ordered a Medium pizza. Good choice for two!");  
                break;  
            case LARGE:  
                System.out.println("You ordered a Large pizza. Great for a group!");  
                break;  
            case EXTRA_LARGE:  
                System.out.println("You ordered an Extra Large pizza. Party time!");  
                break;  
            default:  
                System.out.println("Unknown size.");  
        }  
    }  
}
```

```

    }

    public static void main(String[] args) {
        System.out.println("Subhapreet Patro: 2211CS010547");
        Q2 pizza1 = new Q2(PizzaSize.SMALL);
        pizza1.printPizzaMessage();
        Q2 pizza2 = new Q2(PizzaSize.LARGE);
        pizza2.printPizzaMessage();
    }
}

```

**Output:**

```

Subhapreet Patro: 2211CS010547
You ordered a Small pizza. Perfect for one!
You ordered a Large pizza. Great for a group!

```

**3. Write a Java program that defines a Student class implementing the Comparable<Student> interface and uses Arrays.sort() to sort an array of Student objects by roll number. The program should then print both unsorted and sorted student details.**

**Solution:**

```

import java.util.Arrays;

public class q3 {
    static class Student implements Comparable<Student> {
        int rollNumber;
        String name;

        public Student(int rollNumber, String name) {
            this.rollNumber = rollNumber;
            this.name = name;
        }

        @Override
        public int compareTo(Student other) {
            return Integer.compare(this.rollNumber, other.rollNumber);
        }

        @Override
        public String toString() {

```

```
        return "Roll No: " + rollNumber + ", Name: " + name;
    }
}

public static void main(String[] args) {
    System.out.println("Subhapreet Patro: 2211CS010547");
    Student[] students = {
        new Student(102, "Subham"),
        new Student(101, "Divyansh"),
        new Student(104, "Bharadwaj"),
    };

    System.out.println("Unsorted Student Details:");
    for (Student s : students) {
        System.out.println(s);
    }

    Arrays.sort(students);

    System.out.println("\nSorted Student Details (by Roll Number):");
    for (Student s : students) {
        System.out.println(s);
    }
}
```

**Output:**

```
Subhapreet Patro: 2211CS010547
Unsorted Student Details:
Roll No: 102, Name: Subham
Roll No: 101, Name: Divyansh
Roll No: 104, Name: Bharadwaj
Sorted Student Details (by Roll Number):
Roll No: 101, Name: Divyansh
Roll No: 102, Name: Subham
Roll No: 104, Name: Bharadwaj
```

---

**4. Design a Java class that demonstrates encapsulation by using private fields, public getters/setters, overriding the toString() method, and testing the class functionality via a main method.**

**Solution:**

```
public class q4 {
    static class Person {
        private String name;
        private int age;

        public Person(String name, int age) {
            this.name = name;
            this.age = age;
        }

        public String getName() {
            return name;
        }

        public void setName(String name) {
            this.name = name;
        }

        public int getAge() {
            return age;
        }

        public void setAge(int age) {
            this.age = age;
        }

        @Override
        public String toString() {
            return "Person{name='" + name + "', age=" + age + "}";
        }
    }

    public static void main(String[] args) {
        System.out.println("Subhapreet Patro: 2211CS010547");
        Person p1 = new Person("Subham", 20);
        System.out.println(p1);
        p1.setName("Divyansh");
        p1.setAge(21);
    }
}
```

```
        System.out.println("Updated: " + p1);
        System.out.println("Name: " + p1.getName());
        System.out.println("Age: " + p1.getAge());
    }
}
```

**Output:**

```
Subhapreet Patro: 2211CS010547
Person{name='Subham', age=20}
Updated: Person{name='Divyansh', age=21}
Name: Divyansh
Age: 21
```

- 
- 5. Write a Java program to read the following inputs from the user - Name, Rollno, height and weight. And check whether the height is greater than 170 and weight is less than 60kgs.If these conditions are satisfied the candidate is eligible to become an NCC cadet, otherwise print not eligible**

**Solution:**

```
import java.util.Scanner;

public class q5 {

    public static void main(String[] args) {

        System.out.println("Subhapreet Patro: 2211CS010547");

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Name: ");

        String name = sc.nextLine();

        System.out.print("Enter Rollno: ");

        String rollno = sc.nextLine();

        System.out.print("Enter Height (in cm): ");

        int height = sc.nextInt();
```

```
System.out.print("Enter Weight (in kg): ");

int weight = sc.nextInt();

if (height > 170 && weight < 60) {

    System.out.println("Eligible to become an NCC cadet");

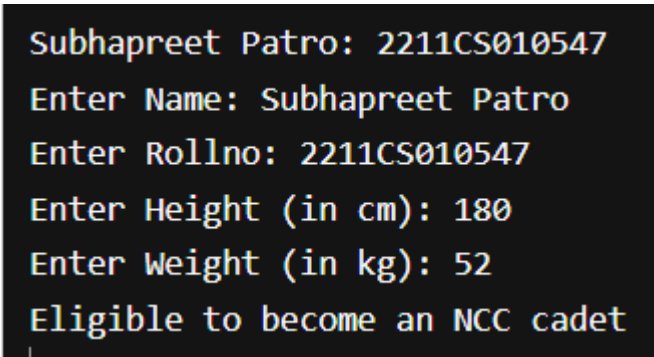
} else {

    System.out.println("Not eligible");

}

}
```

**Output:**

A screenshot of a terminal window with a black background and white text. The output shows the program's execution: 'Subhapreet Patro: 2211CS010547' followed by prompts and user input for Name, Rollno, Height, and Weight, and finally the eligibility result.

```
Subhapreet Patro: 2211CS010547
Enter Name: Subhapreet Patro
Enter Rollno: 2211CS010547
Enter Height (in cm): 180
Enter Weight (in kg): 52
Eligible to become an NCC cadet
```

---

**6. Create classes to model a person with multiple mobile numbers. Practice encapsulation with private fields, getters/setters, using List<Mobile>, and printing combined data in a meaningful format.**

**Solution:**

```
import java.util.ArrayList;

import java.util.List;

public class q6 {

    static class Mobile {
```

```
private String number;

public Mobile(String number) {

    this.number = number;

}

public String getNumber() {

    return number;

}

public void setNumber(String number) {

    this.number = number;

}

@Override

public String toString() {

    return number;

}

}

static class Person {

    private String name;

    private List<Mobile> mobiles;

    public Person(String name) {

        this.name = name;

        this.mobiles = new ArrayList<>();

    }

    public String getName() {

        return name;

    }

}
```



```
public void setName(String name) {
```

```
    this.name = name;
```

```
}
```

```
public List<Mobile> getMobiles() {
```

```
    return mobiles;
```

```
}
```

```
public void addMobile(Mobile mobile) {
```

```
    this.mobiles.add(mobile);
```

```
}
```

```
@Override
```

```
public String toString() {
```

```
    return "Person{name='" + name + "', mobiles='" + mobiles + "'}";
```

```
}
```

```
}
```

```
public static void main(String[] args) {
```

```
    System.out.println("Subhapreet Patro: 2211CS010547");
```

```
    Person p = new Person("Subhapreet");
```

```
    p.addMobile(new Mobile("7569753540"));
```

```
    p.addMobile(new Mobile("9989932867"));
```

```
    p.addMobile(new Mobile("9398054747"));
```

```
    System.out.println(p);
```

```
}
```

```
}
```

**Output:**

```
Subhapreet Patro: 2211CS010547
Person{name='Subhapreet', mobiles=[7569753540, 9989932867, 9398054747]}
```

---

**7. Build a Java application that uses regular expressions to validate email addresses, demonstrating your ability to use Pattern, Matcher, and implement reusable validation logic.**

**Solution:**

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;
import java.util.Scanner;

public class q7 {
    public static boolean isValidEmail(String email) {
        String emailRegex = "^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$";
        Pattern pattern = Pattern.compile(emailRegex);
        Matcher matcher = pattern.matcher(email);
        return matcher.matches();
    }

    public static void main(String[] args) {
        System.out.println("Subhapreet Patro: 2211CS010547");
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter email address to validate: ");
        String email = sc.nextLine();
        if (isValidEmail(email)) {
            System.out.println("Valid email address.");
        } else {
            System.out.println("Invalid email address.");
        }
    }
}
```

**Output:**

```
Subhapreet Patro: 2211CS010547
```

```
Enter email address to validate: 2211cs010547@mallareddyuniversity.ac.in
```

```
Valid email address.
```

**8. Develop a tool to parse simplified server log entries for analysis. Each log entry is a single line string with fields separated by a pipe symbol (|). For example:**

**"2025-07-27|INFO|UserLogin|UserID=12345|Action=LOGIN\_SUCCESS"**

**Your task: write Java code using StringTokenizer to break this log line into its components, extract meaningful information, and present a summary.**

**Solution:**

```
import java.util.StringTokenizer;
```

```
import java.util.Scanner;
```

```
public class q8 {  
    public static void main(String[] args) {  
        System.out.println("Subhapreet Patro: 2211CS010547");  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter log entry: ");  
        String logEntry = sc.nextLine();  
  
        StringTokenizer st = new StringTokenizer(logEntry, "|");  
        String date = st.hasMoreTokens() ? st.nextToken() : "";  
        String level = st.hasMoreTokens() ? st.nextToken() : "";  
        String event = st.hasMoreTokens() ? st.nextToken() : "";  
        String userIdField = st.hasMoreTokens() ? st.nextToken() : "";  
        String actionField = st.hasMoreTokens() ? st.nextToken() : "";  
  
        String userId = userIdField.replace("UserID=", "");  
        String action = actionField.replace("Action=", "");  
  
        System.out.println("--- Log Summary ---");  
        System.out.println("Date: " + date);  
        System.out.println("Level: " + level);  
        System.out.println("Event: " + event);  
        System.out.println("User ID: " + userId);  
        System.out.println("Action: " + action);  
    }  
}
```

**Solution:**

```
Subhapreet Patro: 2211CS010547
Enter log entry: 31-07-25
--- Log Summary ---
Date: 31-07-25
Level:
Event:
User ID:
Action:
```

- 
- 9. Develop a simple console-based calculator used in a learning application. Users should be able to repeatedly choose arithmetic operations (addition, subtraction, multiplication, division, modulus) from a menu, perform calculations on two numbers, and exit when they wish.**

**Solution:**

```
import java.util.Scanner;

public class q9 {
    public static void main(String[] args) {
        System.out.println("Subhapreet Patro: 2211CS010547");
        Scanner sc = new Scanner(System.in);
        while (true) {
            System.out.println("\nCalculator Menu:");
            System.out.println("1. Addition");
            System.out.println("2. Subtraction");
            System.out.println("3. Multiplication");
            System.out.println("4. Division");
            System.out.println("5. Modulus");
            System.out.println("6. Exit");
            System.out.print("Choose an operation (1-6): ");
            int choice = sc.nextInt();
            if (choice == 6) {
                System.out.println("Exiting calculator. Goodbye!");
                break;
            }
            System.out.print("Enter first number: ");
            double num1 = sc.nextDouble();
```

```
System.out.print("Enter second number: ");
double num2 = sc.nextDouble();
switch (choice) {
    case 1:
        System.out.println("Result: " + (num1 + num2));
        break;
    case 2:
        System.out.println("Result: " + (num1 - num2));
        break;
    case 3:
        System.out.println("Result: " + (num1 * num2));
        break;
    case 4:
        if (num2 != 0)
            System.out.println("Result: " + (num1 / num2));
        else
            System.out.println("Error: Division by zero");
        break;
    case 5:
        if (num2 != 0)
            System.out.println("Result: " + ((int)num1 % (int)num2));
        else
            System.out.println("Error: Division by zero");
        break;
    default:
        System.out.println("Invalid choice. Please try again.");
}
}
```

**Output:**

Subhapreet Patro: 2211CS010547

Calculator Menu:

1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Modulus
6. Exit

Choose an operation (1-6): 1

Enter first number: 3

Enter second number: 5

Result: 8.0

Calculator Menu:

1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Modulus
6. Exit

Choose an operation (1-6): 6

Exiting calculator. Goodbye!

---

**10. Develop a simple Library Management System where each Library maintains several Book instances. Books are tightly coupled to their library, and each library can also process user Membership Cards for borrowing books. You decide to use inner classes to encapsulate this logic.**

**Solution:**

```
public class q10 {  
    static class Library {  
        private String name;  
        private Book[] books;  
        private int bookCount;
```

```

private static final int MAX_BOOKS = 10;

public Library(String name) {
    this.name = name;
    this.books = new Book[MAX_BOOKS];
    this.bookCount = 0;
}

public void addBook(String title, String author) {
    if (bookCount < MAX_BOOKS) {
        books[bookCount++] = new Book(title, author);
    } else {
        System.out.println("Cannot add more books.");
    }
}

public void showBooks() {
    System.out.println("Books in " + name + ":");
    for (int i = 0; i < bookCount; i++) {
        System.out.println((i+1) + ". " + books[i]);
    }
}

public void processMembership(String cardNumber, String userName) {
    MembershipCard card = new MembershipCard(cardNumber, userName);
    System.out.println("Membership processed: " + card);
}

class Book {
    private String title;
    private String author;

    public Book(String title, String author) {
        this.title = title;
        this.author = author;
    }

    @Override
    public String toString() {
        return "\"" + title + " by " + author;
    }
}

class MembershipCard {

```

```
private String cardNumber;
private String userName;

public MembershipCard(String cardNumber, String userName) {
    this.cardNumber = cardNumber;
    this.userName = userName;
}

@Override
public String toString() {
    return "Card No: " + cardNumber + ", User: " + userName;
}
}

public static void main(String[] args) {
    System.out.println("Subhapreet Patro: 2211CS010547");
    Library lib = new Library("CRT Central Library");
    lib.addBook("Java Programming", "James Gosling");
    lib.addBook("Effective Java", "Joshua Bloch");
    lib.addBook("Clean Code", "Robert C. Martin");
    lib.showBooks();
    lib.processMembership("MC1001", "Subhapreet");
}
}
```

**Output:**

```
Subhapreet Patro: 2211CS010547
Books in CRT Central Library:
1. 'Java Programming' by James Gosling
2. 'Effective Java' by Joshua Bloch
3. 'Clean Code' by Robert C. Martin
Membership processed: Card No: MC1001, User: Subhapreet
```

---